

ChampionCHIP eXperience

Phase 2 Challenge - Design Submission Guide

This document defines the mandatory deliverables for Phase 2 of the ChampionCHIP eXperience.

Participants must present the architecture, implementation, validation, and physical results of the developed project, including technical documentation, OpenLane results, and a functional demonstration of the processor.

1. Project Overview

Describe the project's overall goal, the architecture used, the main modules implemented, and the system's general operation. Also, explain which functionalities were implemented in the RISC-V core.

2. Supported Instructions

The participant must present the coverage of instructions implemented in the processor, grouped by the category of the RISC-V ISA used in the project.

A summary table containing the quantity of instructions expected by the competition and the quantity effectively implemented in each category must be included.

ISA Coverage Table

Category	Expected Instructions	Implemented Instructions	Coverage (%)
Arithmetic and Logic (Register)	10	XX	XX%
Arithmetic and Logic (Immediate)	9	XX	XX%
Load	5	XX	XX%
Store	3	XX	XX%
Branch	6	XX	XX%
Jump	2	XX	XX%

Upper Immediate	2	XX	XX%
System / Synchronization	3	XX	XX%
Multiplication	4	XX	XX%
CRC	3	XX	XX%
Total	47	XX	XX%

In addition to the summary table, the participant must present the list of instructions implemented in each category

Example

Arithmetic and Logic (Register) (10/10)

ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND

Load (5/5)

LB, LH, LW, LBU, LHU

CRC (3/3)

CRCB, CRCH, CRCW

Finally, a table containing the encoding of all implemented instructions, including opcode, funct3, and funct7 (where applicable), must be presented.

Encoding Example

Instruction	Opcode	Funct3	Funct7
ADD	0110011	000	0000000
SUB	0110011	000	0100000
MUL	0110011	000	0000001
CRCB	0110011	000	1000000

The coverage table will be used by the evaluators to quickly verify the percentage of ISA implementation supported by the processor.

3. Datapath

Present the complete design of the processor's datapath, along with the control unit, showing the flow of data between the main modules of the system.

4. Modules and Testbenches

Describe each developed module and its corresponding testbench. Include the relevant block diagrams generated in ChipInventor to illustrate the processor architecture and module interconnections.

5. OpenLane Results

Present the results obtained for total area and density after the physical implementation flow using OpenLane, as well as the final GDSII image of the project.

6. Firmware Demo

Present waveform images that prove the correct execution of the validation firmware, as well as the output of logs generated by the testbench, proving that the execution reached the end of validation without errors.

Note: The official validation firmware will be made available on the ChampionCHIP platform. The score for this category will be based on the number of validation stages successfully completed by the processor. Successfully completing all validation stages will result in the maximum score for this category.

7. Video Demo

Produce a video demonstrating:

- Project architecture
- Processor operation
- System simulation
- Firmware execution
- OpenLane results
- Final chip layout

The video must be 3 to 4 minutes long and uploaded to YouTube as a public or unlisted video. The video URL must be included in the report.

8. Report Format

The report must contain topics 1 to 6 of this document in PDF format, with a maximum of 20 pages and a maximum file size of 10 MB.

9. Expected Files

The following files must be included in the project submission:

- Report
- GDSII file generated by the OpenLane Synthesis
- GL (gate-level netlist .v) generated by OpenLane, located in results/final/verilog/gl folder
- config.json file used by the OpenLane Synthesis
- All developed code including testbench files
- YouTube URL for the project demonstration video, included in the report

Note 1 - Submission Integrity:

The submitted RTL, testbenches, configuration files, and generated physical-design results must correspond to the same processor implementation. All code submitted by a team must represent the team's own work.

Participants are allowed to use the single-cycle RISC-V processor developed during the ChampionCHIP training as a starting point for their project. Teams may modify and extend this design to develop the required multicycle processor architecture.

Copying, sharing, or reusing code from other participating teams is strictly prohibited. Submitting third-party or externally sourced processor implementations as the team's own work is also prohibited.

Evaluators may reproduce the simulation, synthesis, and physical-design flow using the submitted files and may compare submissions to identify duplicated or substantially similar code. Evidence of unauthorized code sharing or copying may result in penalties or disqualification.

ChampionCHIP
eXperience

Block Guide

ChipInventor RVBL-2

Table of Contents

1. Contextualization	3
1.1 Conventions	3
1.2 References	3
1.3 Acronyms and Abbreviations	3
1.4 Glossary	4
1.5 Notations	4
2. Introduction	7
2.1 The Core	5
2.2 IP Pins Description	5
2.3 Internal Structure	6
2.4 Instruction Set	7
3. RISC-V Core	11
3.1 Datapath of the Core	9
3.2 Control Unit	12
3.3 LSU (Load/Store Unit)	12
4. Memory Mapping	16
4.1 Memory Map	16
4.2 Instruction Memory (IMEM)	16
4.3 Data Memory (DMEM)	17
4.4 Address Decoder	18

1. Contextualization

This document presents the functionalities, details and IP requirements of a 32-bit RISC-V-based microcontroller, with the aim of guiding its use and replication. While in development, the definitions of its functions serve as a specification for the implementation of IP.

1.1 Conventions

Table 1 contains the conventions used in this document.

Table 1. List of Conventions

Convention	Definition
I	Input-only signal
O	Output-only signal
I/O	Input and output signal
R	Read-only access to bit or byte of data
W	Write-only access to bit or byte of data
R/W	Read-and-write access to bit or byte of data

1.2 References

[1] The RISC-V Instruction Set Manual, Volume I; Unprivileged Architecture (2026).

1.3 Acronyms and Abbreviations

Table 2 contains the acronyms and abbreviations used in this document.

Table 2. Acronyms and Abbreviations

Term	Definition
MSB	Most Significant Bit
LSB	Least Significant Bit
ISA	Instruction Set Architecture
CRC	Cyclic Redundant Check
ALU	Arithmetic and Logic Unit

1.4 Glossary

Table 3 contains the terms used in this document.

Table 3. Glossary

Term	Definition
Active low	The signal is activated when it changes the logic level to zero
Active high	The signal is activated when it changes the logic level to one
Asserted	When the discrete signal is in the activation state
Negated	When the discrete signal is in the deactivation state
Pin	External Physical Connection

1.5 Notations

Table 4 contains the notations used in this document.

Table 4. Notations

Notation	Definition
{ a , b }	Concatenation of a and b signals
x'by	A string y of x bits in binary format
x'dy	A string y of x bits in decimal format
x'hy	A string y of x bits in hexadecimal format
Mem[a]	Memory contents stored at address <i>a</i>
[a : b]	A bus formed by the set of all bits between <i>a</i> and <i>b</i> , included, where <i>a</i> is the most significant bit and <i>b</i> is the least significant bit

2. Introduction

The ChipInventor RVBL-2 is a microcontroller built around a RISC-V-based core, being manufactured using the 130nm Sky130 PDK. This guide serves as an architectural reference for using the microcontroller.

2.1 The Core

The microcontroller includes a RISC-V core that implements 32-bit ISA, operating with multiple *clock* cycles per instruction (i.e., *multi-cycle*). This approach allows the sharing of *hardware* resources (such as memory interfaces and arithmetic units) at different stages of execution, optimizing the chip area.

The kernel is compatible with the RISC-V standard specified in [1], implementing the base ISA **RV32I** (for 32-bit integer operations) as well as the standardized **Zmmul** extension (derived from the M extension for multiplication instructions only). It also implements a custom extension to calculate CRC called **Xicrc**.

2.2 IP Pins Description

The IP contains only 2 pins: 1 for synchronization and 1 for (re)initialization. Table 5 describes the interface signals of the block:

Table 5. Description of Pins

Pin Name	Dir.	Reset	Description
rst_i	I	-	Reset signal. Restarts the operation of the IP.
clk_i	I	-	Core <i>clock</i> signal . Guide IP operation.

2.3 Internal Structure

In addition to the RISC-V-based core, IP contains other important components for its functioning. Table 6 lists the set of key features contained in the ChipInventor RVBL-2 microcontroller.

Table 6. IP Feature Set

Resource	Description
Core (RISC-V)	RISC-V core implementing ISA RV32I_Zmmul_Xicrc.
Instruction Memory (IMEM)	Up to 4MB ROM memory.

Data Memory (DMEM)	SRAM memory up to 8kB.
--------------------	------------------------

The RISC-V-based core communicates with the other modules (data memories and instructions) via memory addresses. Therefore, a module for decoding addresses is used to mediate the communication.

Figure 1 contains the high-level diagram of the IP, showing its main internal components and their interconnections:

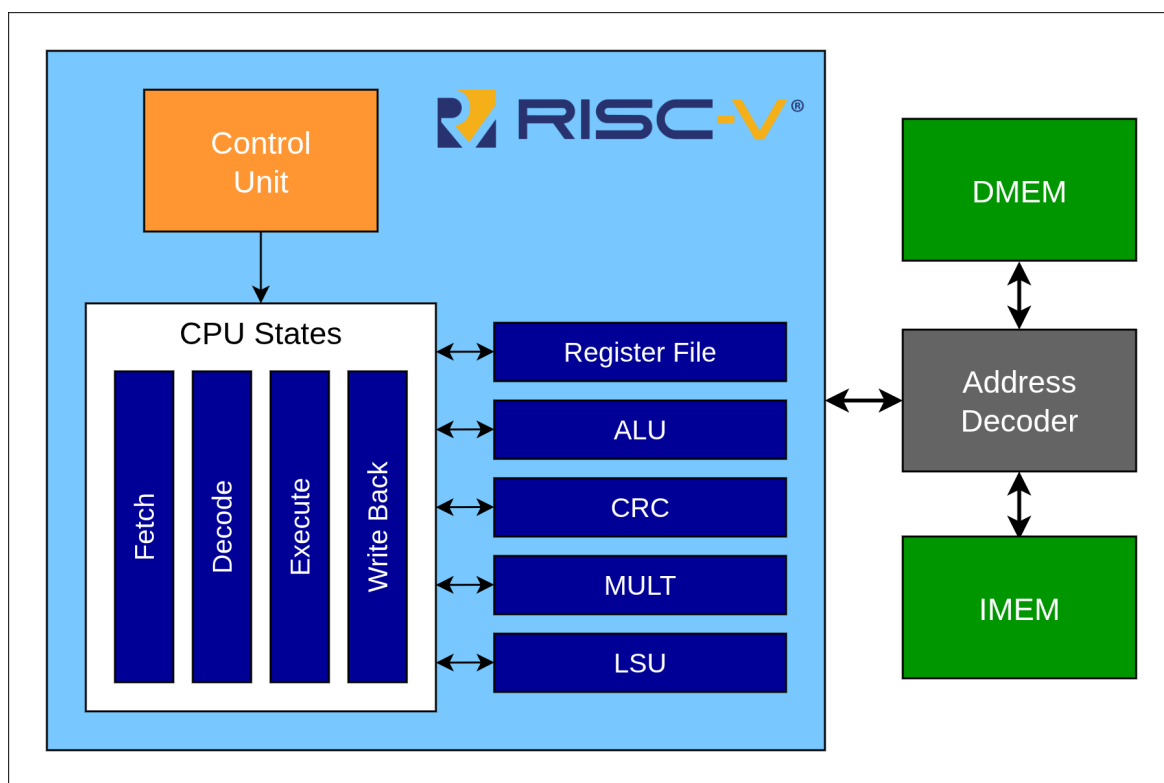


Figure 1. High Level IP Block Diagram

2.4 Instruction Set

As mentioned earlier, the microcontroller's inner core implements the RISC-V architecture's RV32I base instruction set, as well as two additional extensions (Zmmul and Xicrc).

In order to reduce the time required to perform multiplication operations, it is necessary to add a specific hardware block for it. The extension of RISC-V that standardizes this is M, which specifies multiplication and division instructions. Splitting instructions, however, greatly increase the area of the chip. In order to reduce the area used by the kernel, the instructions specified by the Zmmul extension were implemented, which is a subset of the M extension containing only

multiplication instructions. These instructions can be found in Table 7 as well as in chapter **12.3. Zmmul Extension** of the RISC-V Instruction Set Manual [1]:

Table 7. Zmmul Extension Supported Instructions

Opcode	funct3	funct7	Type	Instruction	Operation
0110011	000	0000001	R	mul rd, rs1, rs2	$rd = (rs1_{signed} * rs2_{signed})_{31:0}$
0110011	001	0000001	R	mulh rd, rs1, rs2	$rd = (rs1_{signed} * rs2_{signed})_{63:32}$
0110011	010	0000001	R	mulhsu rd, rs1, rs2	$rd = (rs1_{signed} * rs2_{unsigned})_{63:32}$
0110011	011	0000001	R	mulhu rd, rs1, rs2	$rd = (rs1_{unsigned} * rs2_{unsigned})_{63:32}$

The core is also capable of processing the CRC-16 natively through custom instructions, without the need for a specific peripheral, composing the Xicrc custom extension. This is made possible by the expansive nature of the RISC-V. These instructions can be found in Table 8:

Table 8. Xicrc Extension Supported Instructions

Opcode	funct3	funct7	Type	Instruction	Operation
0110011	000	1000000	R	crcb rd, rs1, rs2	$rd = CRC8(rs1, rs2)_{15:0}$
0110011	001	1000000	R	crch rd, rs1, rs2	$rd = CRC16(rs1, rs2)_{15:0}$
0110011	010	1000000	R	crcw rd, rs1, rs2	$rd = CRC32(rs1, rs2)_{15:0}$

Since these instructions are customized , *the RISC-V assembler does not recognize them, so in order to be used, assembly directives must be used to inform the opcode, funct3 and funct7 fields.* This makes it possible to use the instructions, leaving it to the assembler to translate them into machine code. The directives used are shown in the code below:

```
.macro crcb rd, rs1, rs2
    .insn r 0x33, 0, 0x40, \rd, \rs1, \rs2
.endm

.macro crch rd, rs1, rs2
    .insn r 0x33, 1, 0x40, \rd, \rs1, \rs2
.endm

.macro crcw rd, rs1, rs2
    .insn r 0x33, 2, 0x40, \rd, \rs1, \rs2
.endm
```

3. RISC-V Core

This section covers the internal organization of the RISC-V core, describing the main units that compose it. Many different organizations make it possible to implement the RISC-V architecture specified in [1] in a correct manner, allowing its normal use.

Therefore, this section seeks to bring the chosen implementation for this kernel, which involves a state machine using multiple *clock* cycles to control the kernel operation, as well as implementation of the extensions discussed above and other important modules for the correct implementation of the ISA RV32I instructions.

3.1 Core Datapath

It holds the core units of the kernel (shown in Figure 1), forming the path through which data travels during the execution of instructions.

3.1.1 ALU

It is the unit responsible for performing all the mathematical and logical operations of the core: additions, subtractions, logical operations, and bit offsets. In addition to calculating results for logical/arithmetic instructions, it is reused to calculate memory addresses (in loads/stores) and deviation targets (branches/jumps).

It is primarily composed of several combinational blocks that perform operations on inputs A and B. The result of each block is sent to a multiplexer that selects the result that should be passed on to the output based on a signal sent by the control unit. The values for this check mark can be seen in Table 9.

Table 9. ALU Module Multiplexers Selection Values

Value	Operation	Description
4'h0	PASS_B	$Q = B$
4'h1	ADD	$Q = A + B$
4'h2	SUB	$Q = A - B$
4'h3	AND	$Q = A \& B$
4'h4	OR	$Q = A B$
4'h5	XOR	$Q = A \wedge B$
4'h6	SLL	$Q = A \ll B[4:0]$
4'h7	SRL	$Q = A \gg B[4:0]$ (Logical Shift)

4'h8	MRS	$Q = A \ggg B[4:0]$ (Arithmetic Shift)
4'h9	SLT	if $(A < B)$ $Q = 1$, else $Q = 0$
4'hA	SLTU	if $(A_{\text{unsigned}} < B_{\text{unsigned}})$ $Q = 1$, else $Q = 0$

3.1.2 Multiplier

It is the unit responsible for performing the core multiplication operations.

It is composed of multiplexers that select which inputs should be sent to the block that will perform the multiplication (64 bits), as well as another multiplexer that selects which part of the final result should be sent to the output (the lower or upper 32 bits).

The values of the selection signals of these multiplexers can be seen in Table 10.

Table 10. MULT Module Multiplexers Selection Values

Value	Operation	Description
4'h0	MUL	$rd = (rs1_{\text{signed}} * rs2_{\text{signed}})_{31:0}$
4'h1	MULH	$rd = (rs1_{\text{signed}} * rs2_{\text{signed}})_{63:32}$
4'h2	MULHSU	$rd = (rs1_{\text{signed}} * rs2_{\text{unsigned}})_{63:32}$
4'h3	MULHU	$rd = (rs1_{\text{unsigned}} * rs2_{\text{unsigned}})_{63:32}$

According to the instruction to be executed, signs A and B with or without sign must be selected to be multiplied. In addition, the bottom or top 32 bits of the result must be selected.

The multiplier then uses a signal from the control unit to select the correct values in the three multiplexers (A, B and result).

3.1.3 CRC

Unit responsible for processing the 16-bit CRC internally to the core, without latency. It consists of three blocks, each of which performs the CRC with a part of the inputs (8, 16 or 32 bits) and a multiplexer, which selects the final value of the blocks and passes it on to the output, based on a selection signal coming from the control unit.

The values of the selection signals of these multiplexers can be seen in Table 11.

Table 11. Values of Internal CRC Module Multiplexer

Value	Operation	Description
4'h0	CRCB	$rd = \text{CRC8}(rs1, rs2)_{15:0}$
4'h1	CRCH	$rd = \text{CRC16}(rs1, rs2)_{15:0}$

4'h2	CRCW	rd = CRC32(rs1, rs2) _{15:0}
------	------	--------------------------------------

According to the instruction to be executed, parts of signals A and B must be selected to compute the 16-bit CRC.

3.1.4 RISC-V Registers

The RISC-V core has 34 registers: 32 general purpose registers (in the register bank) at the user's disposal, the PC (program counter) and the IR (instruction register).

The **register bank** holds temporary CPU data. Contains 32 32-bit general-purpose registers (x0 to x31). The x0 register is hardwired (fixed) at zero and cannot be modified. Allows simultaneous asynchronous reading of the rs1 and rs2 operands, and synchronous writing of the rd target in the *Write Back* stage. The register selection signals are extracted directly from the instruction, and the *write enable* signal is sent by the control unit

The **program counter** (PC) tracks the execution stream while maintaining the 32-bit address of the current instruction. It is updated sequentially (PC+4) with each *clock* cycle or loaded with new addresses derived from flow control instructions (*branch* and *jump*). The RISC-V core uses the PC value to fetch the next instruction to execute.

The **instruction register** (IR) holds the 32 bits referring to the current instruction to be executed, fetched earlier in the *Fetch cycle*. Its contents are read by the other blocks and decoded to allow the correct execution of the instruction.

3.1.5 Immediate Extension

Module responsible for extracting and formatting the immediate values embedded in the RISC-V instructions, being completely combinational. Therefore, it decodes the instructions of type I, S, B, U, and J based on their *opcode* and extracts their immediate.

Therefore, it takes the bits scattered throughout the instruction (based on the formatting of the specific type of the instruction) and performs the signal extension to generate a valid 32-bit operand for the ALU.

3.1.6 Branch Comparator

Unit responsible for executing the conditional hop (or *branch*) statements. This module compares the rs1 and rs2 registers coming from the register bank based on the *branch* statement (selected by the **funct3** field of the statement). The module also needs to worry about whether the comparison takes into account the signal or not.

Being a combinational module, it needs to obtain the input signals, along with the selection, make the desired comparison and inform whether the jump will be taken or not.

3.2 Control Unit

Component responsible for controlling the operation of the RISC-V core, being its most critical component. To do so, it internally implements a Finite State Machine (FSM) that controls the datapath, signaling which units should be used and how they should be used.

Thus, the internal FSM divides the execution into the following phases, each corresponding to a *clock cycle*:

- **Fetch:** Reads the instruction from the ROM using the current value of the PC and saves it to the instruction register (IR).
- **Decode:** Reads the operands from the Register Bank and decodes the instruction to determine the next states.
- **Execute:** ALU takes action to perform the main calculation of the instruction, calculate the effective memory address, or evaluate the condition of a deviation (Branch).
- **Write Back:** Saves the final result (coming from the ALU or RAM via LSU) in the destination register in the Register Bank.

The control unit then receives the *opcode* from the instruction and emits signals that will control the datapath, allowing it to be executed.

3.3 LSU (Load/Store Unit)

This module provides a direct interface between the RISC-V core and the memories. These devices can be accessed by the RISC-V core through addresses, as commented in section **4. Memory Mapping**. These addresses map internal positions to devices that can be read or written by the kernel.

Therefore, LSU has two main functions: managing **data read** (or *load*) and **write** (or *store*) operations.

3.3.1 Load Instructions

The read operation consists of the RISC-V core obtaining the value of a position using its memory address and placing it in an internal register. The RISC-V kernel, being a 32-bit architecture, works primarily with 4-byte words, where the addresses reference 4-byte words, as this is the size of the kernel's registers. The core, however, can access "chunks" of this word (of 1 or 2 bytes).

Listed below are the *RISC-V* loading instructions:

- **lw**: Reads a 4-byte word.
- **lh**: Reads a 2-byte word (with sign)
- **lb**: Reads a 1-byte word (with a sign).
- **lhu**: Reads a 2-byte word (unsigned).
- **lbu**: Reads a 1-byte word (unsigned).

Instructions that access less than 4 bytes need to choose how they will populate the rest of the register. The **lh** and **lb** instructions fill the rest of the register bits with the signal bit (MSB) value of the read value (signal extension). In this way, when reading a value of negative 1 byte (with bit 7 equal to 1) the other 3 bytes of the register will be filled with value 1, keeping the negative value.

The **lhu** and **lbu** instructions fill the rest of the register's bits with zero (extension of zeros). Thus, when reading a value of 1 byte that has no signal (bit 7 should not represent a signal) the other 3 bytes of the register will be filled with a value of 0, keeping the full value.

Table 12 provides an example of data memory to demonstrate the functionality of the different *load* instructions. The first column has the address of each byte of the DMEM (shown in the third column), while the second column has the address of each word (defined by the module).

Table 12. Example of DMEM positions

Absolute Address	DMEM Real Address	Contents
0x10010000	0x0000	0xF1
0x10010001		0xF2
0x10010002		0xF3
0x10010003		0xF4
0x10010004	0x0004	
...		

Below is the result after loading the value into the 0x10010000 address using **lbu** (loads 1 byte of 0x10010000, filling the rest of the register with zeros):

00	00	00	F1
3	2	1	0

Below is the result after loading the value to the 0x10010000 address using **lb** (loads 1 byte of 0x10010000, extending the 0xF1 signal to the rest of the register):

FF	FF	FF	F1
3	2	1	0

Below is the result after loading the value into the 0x10010000 address using **lhu** (loads 2 bytes from 0x10010000, filling the rest of the register with zeros):

00	00	F2	F1
3	2	1	0

Below is the result after loading the value into the 0x10010000 address using **lh** (loads 2 bytes from 0x10010000, extending the 0xF2 signal to the rest of the register):

FF	FF	F2	F1
3	2	1	0

Below is the result after loading the value into the 0x10010000 address using **lw** (loads 4 bytes from 0x10010000):

F4	F3	F2	F1
3	2	1	0

Therefore, LSU must control the *load operation*, establishing which part of the word read from memory will actually be passed to the register.

For example, when reading the 0x10010002 address, only the 0xF3 byte of the word read (in the case 0xF4F3F2F1) will be placed in the register (which should then have the value 0x000000F3). The LSU should then reposition the byte correctly. The bottom 2 bits of the address are used to tell you which byte is being read.

It is worth remembering that **lw** is aligned at 4 bytes, **lh** and **lhu** at 2 bytes, and **lb** and **lbu** at 1 byte.

3.3.2 Store Instructions

The write operation consists of the RISC-V core writing the value of an internal register to a position using its memory address. As with loading, the kernel can write only "parts" of the word from memory.

Listed below are the *RISC-V* store instructions:

- **sw**: Write a 4-byte word.
- **sh**: Write a 2-byte word.
- **sb**: Write a 1-byte word.

These instructions then provide RISC-V with the ability to write an entire word into memory (with **sw**) or smaller parts (with **sh** and **sb**). Unlike *load* instructions, there is no need to extend the value, since, as much as the memory is divided into 4-byte words, each byte of this word is addressable.

Therefore, LSU must control the *store* operation, establishing which part of the memory word should be modified. LSU then emits a signal called a **4-bit byte write**, where each one defines which of the four bytes of the word will be written. In addition, you must position the dice to be written in the desired position of the word. It is worth remembering that **sw** is aligned at 4 bytes, **sh** at 2 bytes, and **sb** at 1 byte.

For example, to write the 0x12 data in the 0x10010002 address (in Table 12) the LSU must inform that the third byte of the word in the 0x0000 position will be written (with a *write byte* equal to 01002), positioning the data correctly (becoming 0x00120000). Thus, the DMEM will write only the third byte!

3.3.3 LSU Interface

Handles access of full words (32 bits), half words (16 bits) and bytes (8 bits), applying the appropriate sign or zero extension in Load operations.

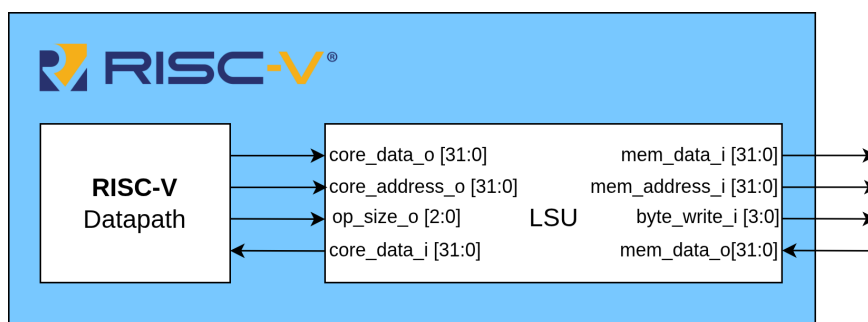


Figure 2. Block diagram of the relationship between the core and LSU

4. Memory Mapping

This section covers IP memory mapping, as well as the operation of memory equipment and the address decoder, which implements the mapping of device memory addresses.

4.1 Memory Map

This device contains the mapped devices to memory addresses located in a 4 GB address space. Table 13 describes the locations of the devices within this memory space.

Table 13. System Memory Map

Base Address	Description
0x00400000	Instruction Memory (IMEM) (4 MB)
0x10010000	Data Memory (DMEM) (8 kB)

4.2 Instruction Memory (IMEM)

This memory contains the program (or *firmware*) to be read and executed by the RISC-V core.

The IMEM has a simple implementation, being only a **ROM** containing the instructions to be executed. Upon receiving the memory address to be accessed and a read signal (*output enable*), the 32-bit instruction that was in that position is received by the kernel to be executed.

Table 14. Example of Firmware in IMEM

Address	Contents
0x00400000	0x00842283
0x00400004	0xf0000437
...	...
0x004000A0	0x00032283
...	...
0x00400100	0x0000000A
...	...

In addition to holding instructions, the IMEM can maintain **constant values** that can be read by the CPU while running a program. For example, the

0x004000A0 : 0x00032283 : **LW** T0, 0(T1)

is located at IMEM, at the address 0x004000A0. Therefore, when the CPU sends this address in the seek cycle, it will receive the value 0x00032283 (hexadecimal equivalent to the instruction stored in memory), which corresponds to the *load instruction*. When executing, it will use **t1** as a memory pointer, and assuming that **t1** has a value 0x00400100 the IMEM will be accessed again, but not to read an instruction, but rather a constant value, which according to Table 14 is the value 10.

The IMEM also contains addresses aligned in 4 bytes, that is, with the bottom two bits always zero (for example 0x00400000, 0x00400004, 0x00400008, etc.).

4.3 Data Memory (DMEM)

This memory contains the data used during the execution of the program by the RISC-V core, the **variables**. It is in the DMEM memory that the variables are kept:

- **Global:** In a part of memory called **.data**.
- **Uninitialized:** In a portion of memory called **.bss**.
- **Local** (or temporary): In a part of memory called the **stack**.
- **Dynamics:** In a part of memory called **the heap**.

DMEM is implemented using SRAM memory cells with synchronous read and write. Thus, when performing a write (via **sw**, **sh** and **sb** instructions) or a read (via **lw**, **lh**, **lb**, **lhu** and **lbu** instructions) **the operation will only be completed after a clock cycle**.

DMEM, internally, addresses 4-byte words instead of 1-byte, that is, it contains addresses aligned in 32 bits, ignoring the bottom two bits of the address (e.g., 0x10010000, 0x10010004, 0x10010008, etc.). Even so, the memory must allow the independent writing of each byte of these 4-byte words (via **sh** and **sb** statements), as required by section 3.3. **LSU (Load/Store Unit)** of the RISC-V specification [1].

To make this possible, the memory maintains *n* 4-byte words, where *n* is the total size of the memory in bytes divided by 4. Thus, a 256-byte memory contains 64 words of 4 bytes, for example. The kernel then executes **sh** (*store half*, writes 2 bytes) or **sb** (*store byte*, writes 1 byte) requests (via **LSU bw_o**) that parts of a word can be written. Memory must then use this signal to select which bytes of the word memory to write.

4.4 Address Decoder

This component acts as an intermediary between the RISC-V core and the memory devices, all accessed through memory addresses (a strategy known as MMIO) and *load* and *store* instructions, isolating the address space of multiple devices.

Its operation is based on receiving a memory address from the RISC-V core and verifying which device is mapped to that address, selecting it for the *load* or *store* operation, routing the necessary signals according to the device (described below).

Figure 3 contains the block diagram of the address decoder, showing its main internal components and their interconnections:

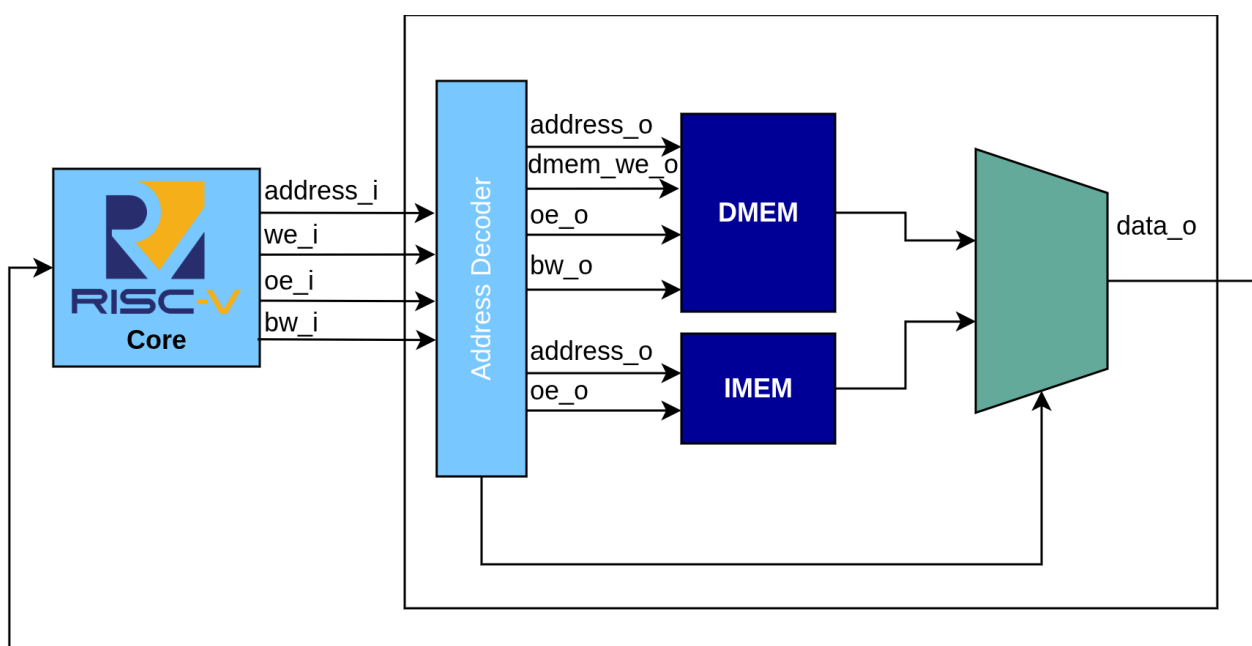


Figure 3. Address Decoder Block Diagram

The core emits four different control signals (in addition to the data input and output buses), which must be routed to the devices by the decoder:

- It sends the **address signal** (coming from the address_o port of the core) to all devices, ignoring the bottom two bits (alignment at 4 bytes).
- Emits the **write signal** (*Write Enable*, coming from the we_o port of the core) to all devices, EXCEPT IMEM memory, which cannot be written.
- It emits the **read signal** (*Output Enable*, coming from the port oe_o the core) to all devices.
- It sends the **Byte Write signal** from the bw_o port of the kernel ONLY to the DMEM memory in order to select the bytes to be written.

That said, the module must route the following signals from the core to all devices: **address_o** (with the exception of the bottom 2 bits), **bw_o** (for DMEM only), **we_o** (with the exception of IMEM), and **oe_o**, as shown in Figure 3.

In addition, the module must parse the address sent by the core, based on the addresses of each device, as shown in Table 13, and select which of the modules' output data will be sent to the core, via a selection signal and a multiplexer.



ChampionCHIP eXperience